

Semester	B.E. Semester VII – Computer Engineering
Subject	R Programming
Subject Professor In- charge	Prof. Sanjeev Dwivedi
Assisting Teachers	Prof. Sanjeev Dwivedi
Laboratory	M-411B

Student Name	Sneha Gadhari
Roll Number	23102B0055
Grade and Subject Teacher's Signature	

Experiment Number	08
Experiment Title	High performance Big Data Analytics using R

Resources / Apparatus Required	Hardware: Computer system	Software: Colab
--------------------------------------	---------------------------------	--------------------

Objective	The objective of this experiment is to build a high-performance and scalable analytics workflow in R on the NYC Yellow Taxi Trip dataset, covering about 9.4 million trips from January to March 2023. It compares functional, vectorized, data.table, sequential and parallel implementations using execution time, memory use and scalability. It also extracts demand, fare, route and payment patterns from the data.
Description	The workflow downloads the official TLC trip files (Parquet) and the Taxi Zone Lookup Table directly from the TLC links. It then runs seven stages: 1. Data preparation 9,384,487 raw trips were imported, reading only the 11 required columns to save memory. - The raw data had invalid records: dates from 2001, negative fares, 135,488 zero-distance trips and 236,179 missing passenger counts. - Cleaning removed duplicates, trips outside the study period, extreme durations, distances, fares and totals, and invalid passenger or payment codes. This left 8,763,337 trips (93.38% retained). - Hour, day, weekday and month were extracted. The raw timestamp columns were dropped, which reduced memory from 787.6 MB to 702 MB. - Pickup and drop-off zones and boroughs were joined using data.table merges, with no unmatched pickup zones. 2. Transportation analytics - Demand peaks at 6 PM (628,711 trips) and is lowest at 4 AM (44,683 trips). - Thursday is the busiest day (15.96% of trips) and Monday the quietest (12.17%). - March is the busiest month (3.17M trips, \$89.5M revenue), against 2.87M trips in January and 2.72M in February. - The afternoon period earns the most revenue (\$90.8M). Night trips have the highest average fare (\$19.29). - The most frequent routes are short Upper East Side hops. The highest-revenue routes are airport routes, led by JFK to outside NYC (\$1.74M). - Distance and fare are strongly related (correlation 0.964, $R^2 = 0.931$, about \$3.71 per mile on top of \$6.01). Fare per mile falls from \$10.87 on trips under 1 mile to \$3.67

on trips over 20 miles.

- Credit card is the dominant payment method in every borough. Manhattan is 82.75% credit card and 16.47% cash, and Queens has a higher cash share (21.58%).
- Additional patterns:
 - Credit-card tips average 25.17% of fare. Cash tips are recorded as 0, because cash tips are not captured in the data.
 - Airport trips are about 10% of all trips but average a \$55.70 fare, against \$14.49 for other trips.
 - Average speed is lowest in the afternoon (about 10 mph) and highest at 5 AM (about 20.7 mph).

3. Functional vs vectorized vs data.table

- Average fare per hour (full 8.76M rows):
 - lapply and purrr::map took about 1.24 s.
 - tapply took about 0.66 s.
 - Vectorized rowsum took about 0.35 s.
 - data.table took about 0.32 s.
- Memory allocation for the lapply and map versions was 1.67 GB, against about 230 to 250 MB for the vectorized and data.table versions.
- Row-wise tip percentage (100,000 rows):
 - apply took about 1,170 ms.
 - map2 took about 254 ms.
 - Vectorized ifelse took about 3 ms.
 - data.table fifelse took about 1.8 ms.
 - data.table was about 650 times faster than apply.

4. Parallel computing

- The data was split into 21 month × weekday partitions.
- A bootstrap regression (200 resamples per partition) ran with foreach sequentially (29.88 s) and in parallel with doParallel on 2 cores (28.15 s).
- Speedup was 1.06× and efficiency 53.1%. Results were identical in both runs.
- The small gain is due to limited cores, the cost of copying partitions to workers, and uneven partition sizes.

5. Benchmarking and scalability

- Grouped mean on 1 million rows: aggregate 696 ms, split + lapply 560 ms, tapply 83 ms, data.table 48 ms. data.table was about 14 times faster than aggregate.

- Scalability test: as rows grew from 100,000 to 8.76M, lapply rose from 0.010 s to 1.224 s, while data.table rose from 0.004 s to 0.262 s.
- All results were collected in one comparison table with times relative to the fastest method.

6. Visualization

- Trips by hour, demand by weekday, monthly demand, average fare by hour and daily revenue.
- Top pickup and drop-off zones, most frequent and highest-revenue routes.
- Payment distribution (overall and by borough) and distance vs fare. All plots use ggplot2 with titles, axis labels, units and legends.

Output

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc	trigger	(Mb)	max	used	(Mb)
Ncells	1275851	68.2	2515760	134.4	2155643	115.2		
Vcells	94360666	720.0	303249923	2313.7	303182353	2313.1		

Memory: 702 Mb

A data.table: 6 × 14

passenger_count	trip_distance	PULocationID	DOLocationID	payment_type	fare_amount	tip_amount	tolls_amount	total_amount	duration_min	hour	day	wday
<int>	<dbl>	<int>	<int>	<fct>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<int>	<fct>
1	0.97	161	141	Cash	9.3	0.00	0	14.30	8.433333	0	1	Sun
1	1.10	43	237	Credit card	7.9	4.00	0	16.90	6.316667	0	1	Sun
1	2.51	48	238	Credit card	14.9	15.00	0	34.90	12.750000	0	1	Sun
1	1.43	107	79	Credit card	11.4	3.28	0	19.68	10.833333	0	1	Sun
1	1.84	161	137	Credit card	12.8	10.00	0	27.80	12.300000	0	1	Sun
1	1.66	239	143	Credit card	12.1	3.42	0	20.52	10.450000	0	1	Sun

A data.table: 4 × 4

period	trips	avg_fare	revenue
<chr>	<int>	<dbl>	<dbl>
Afternoon 12-17	3219714	19.22	90781225
Evening 18-23	2963142	18.29	82422126
Morning 06-11	1919047	18.26	50343801
Night 00-05	661434	19.29	18613794

A data.table: 7 × 3

wday	avg_fare	revenue
<fct>	<dbl>	<dbl>
Sun	19.82	31454294
Mon	19.55	30747386
Tue	18.41	34274356
Wed	18.49	36799564
Thu	18.75	39042508
Fri	18.49	37810420
Sat	17.68	32032418

A data.table: 15 × 3

	route	trips	avg_fare
	<chr>	<int>	<dbl>
	Upper East Side South -> Upper East Side North	61219	8.65
	N/A -> N/A	54860	19.09
	Upper East Side North -> Upper East Side South	52065	9.29
	Upper East Side North -> Upper East Side North	39233	6.72
	Upper East Side South -> Upper East Side South	38218	7.16
	Midtown Center -> Upper East Side South	27368	9.78
	Upper East Side South -> Midtown Center	26708	9.82
	Midtown Center -> Upper East Side North	24896	13.66
	Lincoln Square East -> Upper West Side South	23516	8.02
	Upper West Side South -> Upper West Side North	23191	7.18
	Lenox Hill West -> Upper East Side North	22602	8.73
	Upper West Side South -> Lincoln Square East	22502	7.71
	Upper East Side North -> Midtown Center	20895	14.33

Correlation (distance, fare): 0.964
(Intercept) trip_distance
6.011 3.712
R-squared: 0.9311

A data.table: 6 × 4

dist_band	trips	avg_fare	fare_per_mile
<fct>	<int>	<dbl>	<dbl>
0-1	1935842	7.35	10.87
1-2	2937956	11.32	7.81
2-5	2431616	18.43	6.24
5-10	737998	33.87	4.72
10-20	629334	61.41	4.14
20+	90591	87.66	3.67

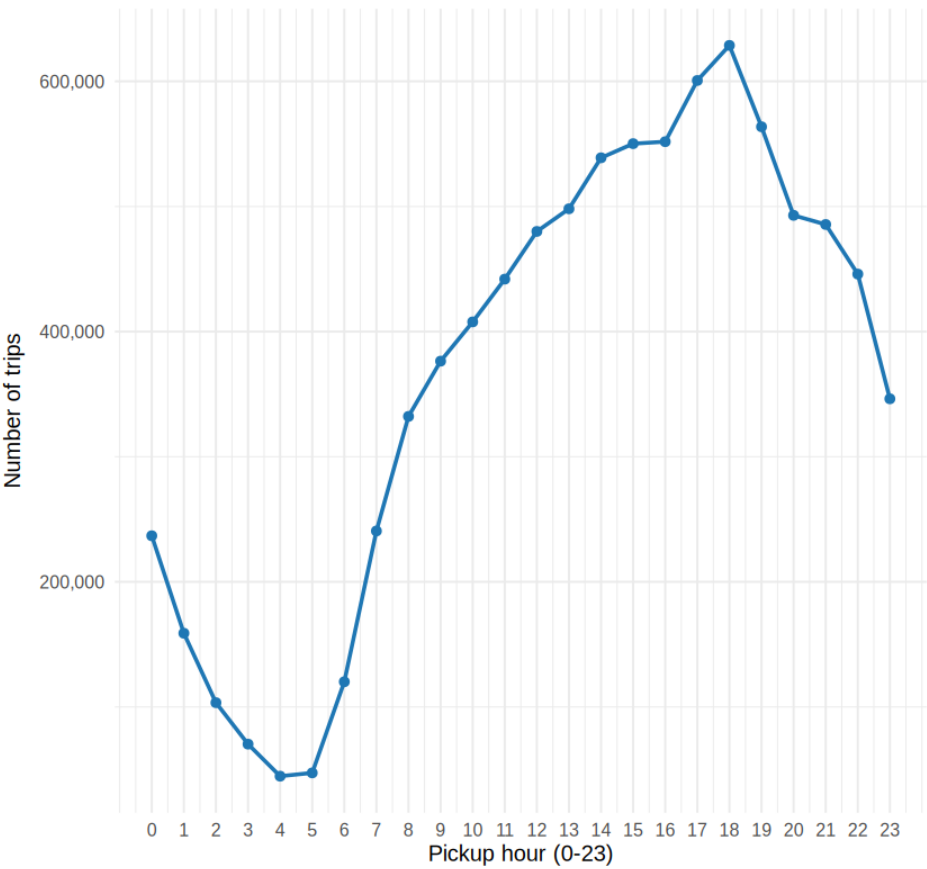
Unit: milliseconds

expr	min	lq	mean	median	uq	max	neval
lapply_functional	1208.8115	1216.0791	1351.9889	1243.0967	1319.6308	1945.9994	10
purrr_map_functional	1191.4570	1232.7914	1359.1911	1239.7130	1461.8569	1870.3976	10
vectorized_rowsum	322.9048	338.4086	388.4486	348.6068	463.0466	521.2789	10
base_tapply	619.5152	647.6458	704.3504	659.0021	692.8872	1098.2473	10
data_table	270.2638	320.0911	353.2131	324.8362	344.0362	535.5666	10

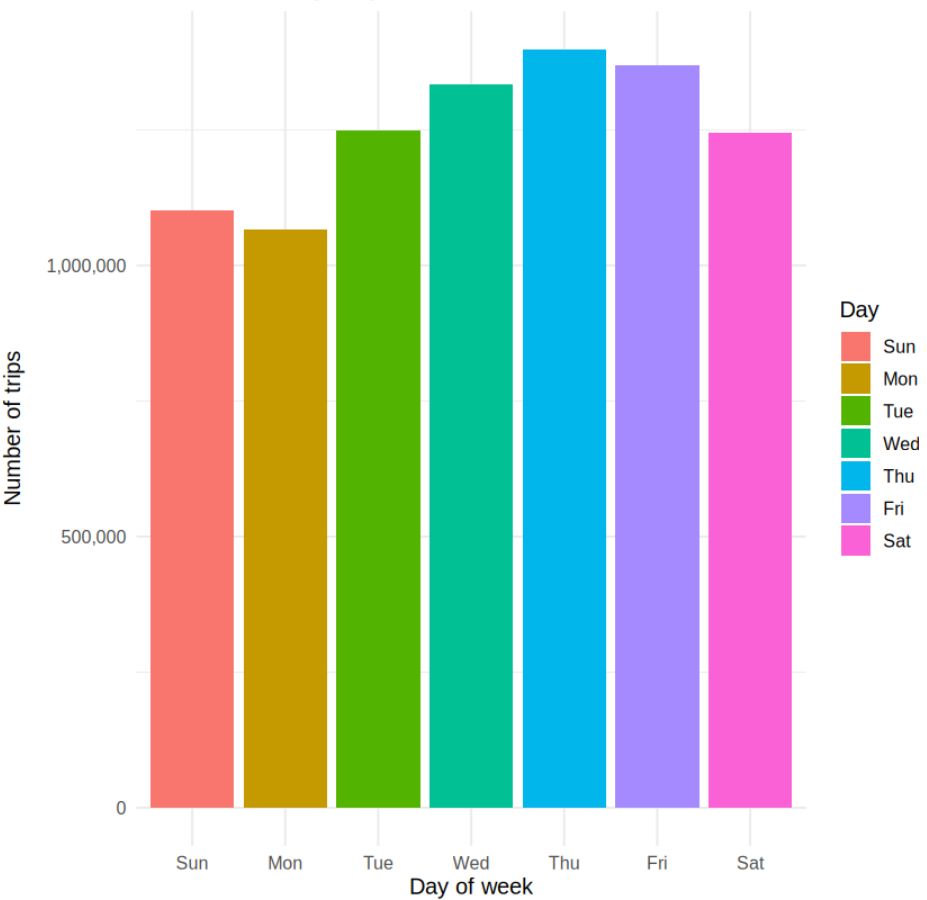
A data.table: 1 × 5				
sequential_s	parallel_s	cores	speedup	efficiency_pct
<dbl>	<dbl>	<int>	<dbl>	<dbl>
29.88	28.15	2	1.06	53.1

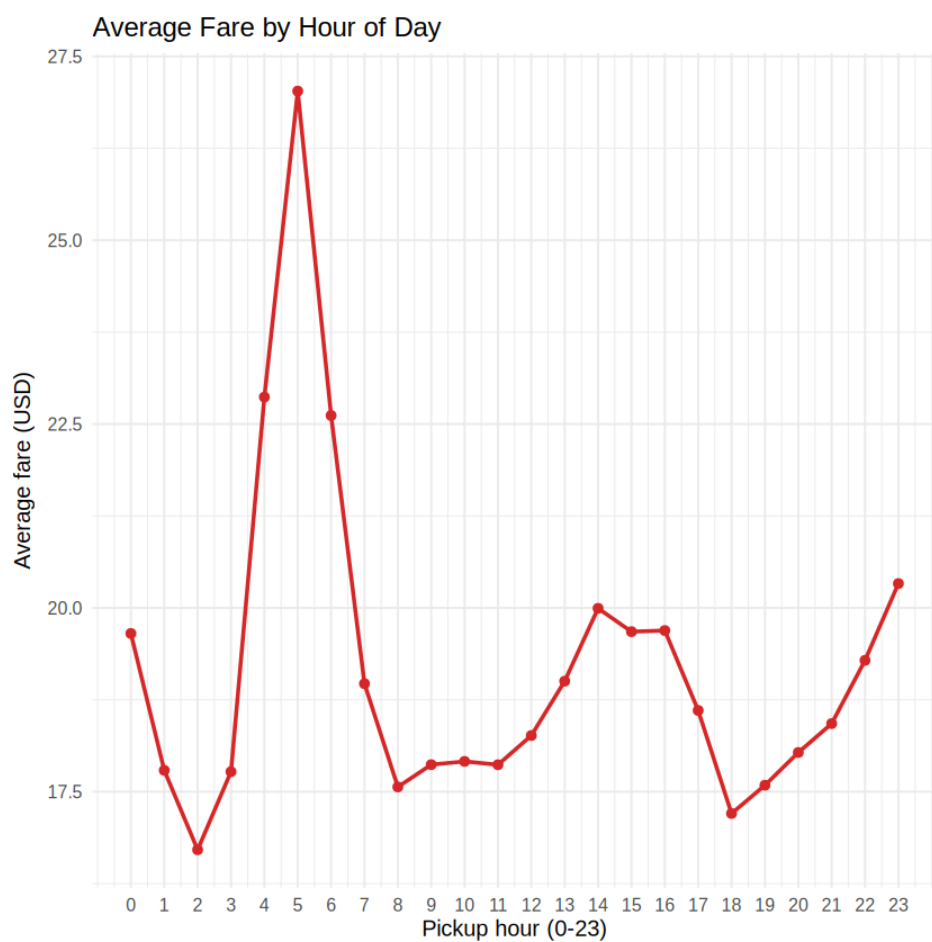
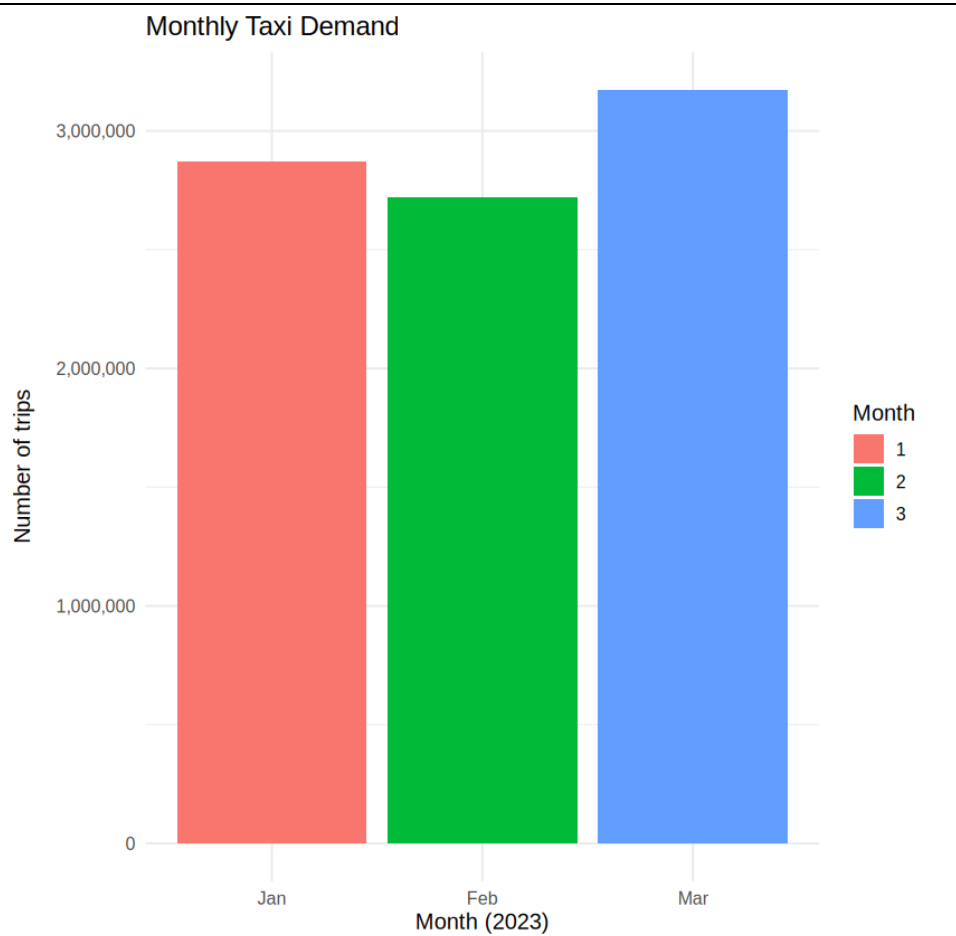
A data.table: 15 × 4				
task	method	median_ms	rel_to_fastest	
<chr>	<chr>	<dbl>	<dbl>	
A: Avg fare per hour (9M rows)	lapply_functional	1243.10	3.8	
A: Avg fare per hour (9M rows)	purrr_map_functional	1239.71	3.8	
A: Avg fare per hour (9M rows)	vectorized_rowsum	348.61	1.1	
A: Avg fare per hour (9M rows)	base_tapply	659.00	2.0	
A: Avg fare per hour (9M rows)	data_table	324.84	1.0	
B: Row-wise tip % (100k rows)	apply_functional	1169.50	653.4	
B: Row-wise tip % (100k rows)	purrr_map2_functional	254.10	142.0	
B: Row-wise tip % (100k rows)	vectorized_ifelse	3.06	1.7	
B: Row-wise tip % (100k rows)	data_table_ifelse	1.79	1.0	
C: Grouped mean (1M rows)	base_aggregate	695.90	14.4	
C: Grouped mean (1M rows)	base_tapply	83.33	1.7	
C: Grouped mean (1M rows)	functional_split_lapply	560.03	11.6	
C: Grouped mean (1M rows)	data_table	48.25	1.0	
Sequential vs parallel (bootstrap, 21 partitions)	sequential_foreach	29876.00	1.1	

NYC Yellow Taxi Trips by Hour of Day
January to March 2023

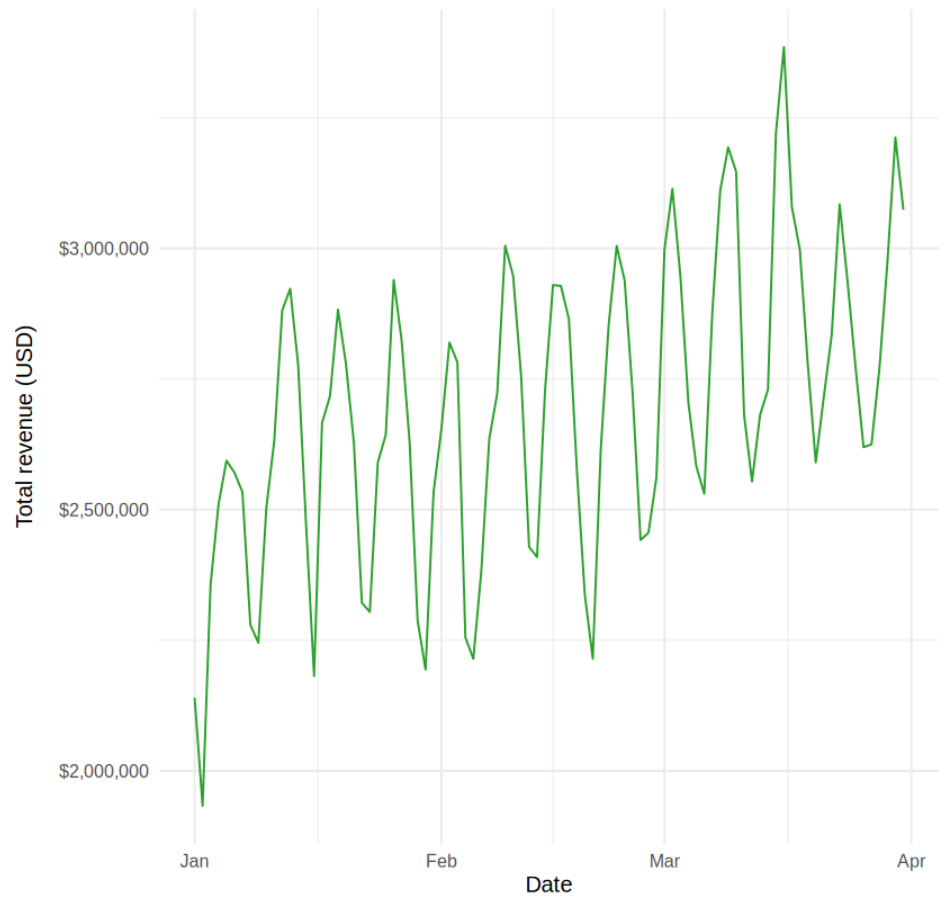


Taxi Demand by Day of Week

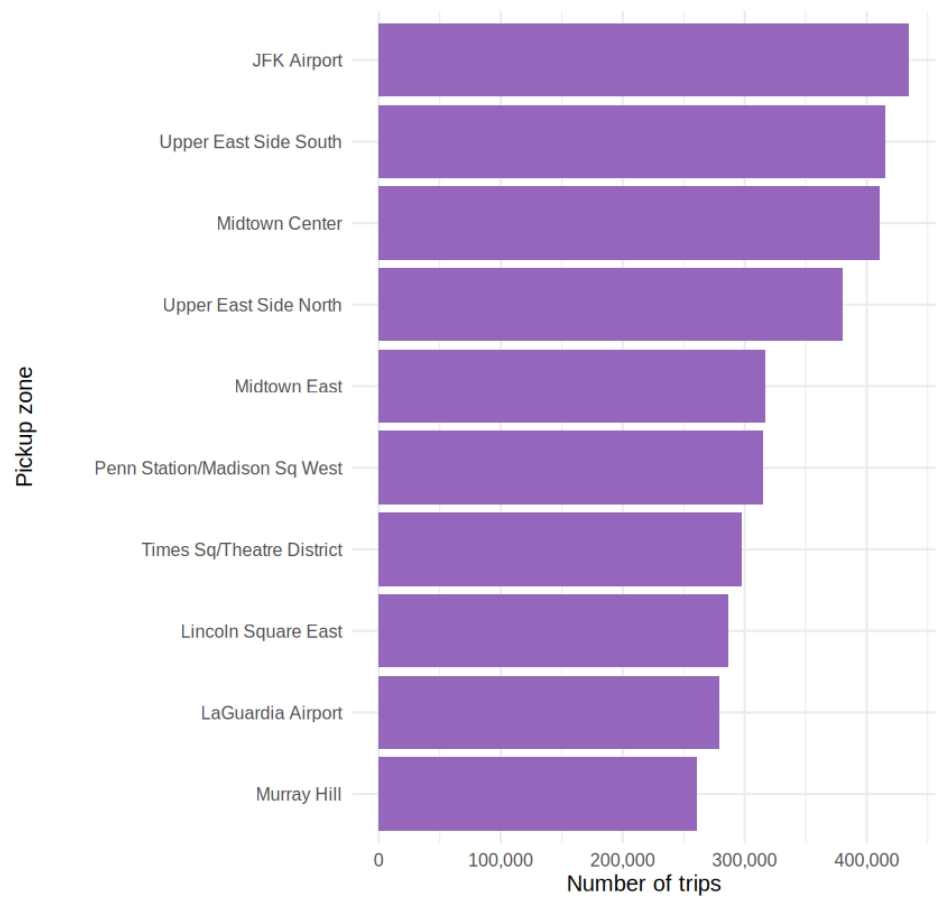


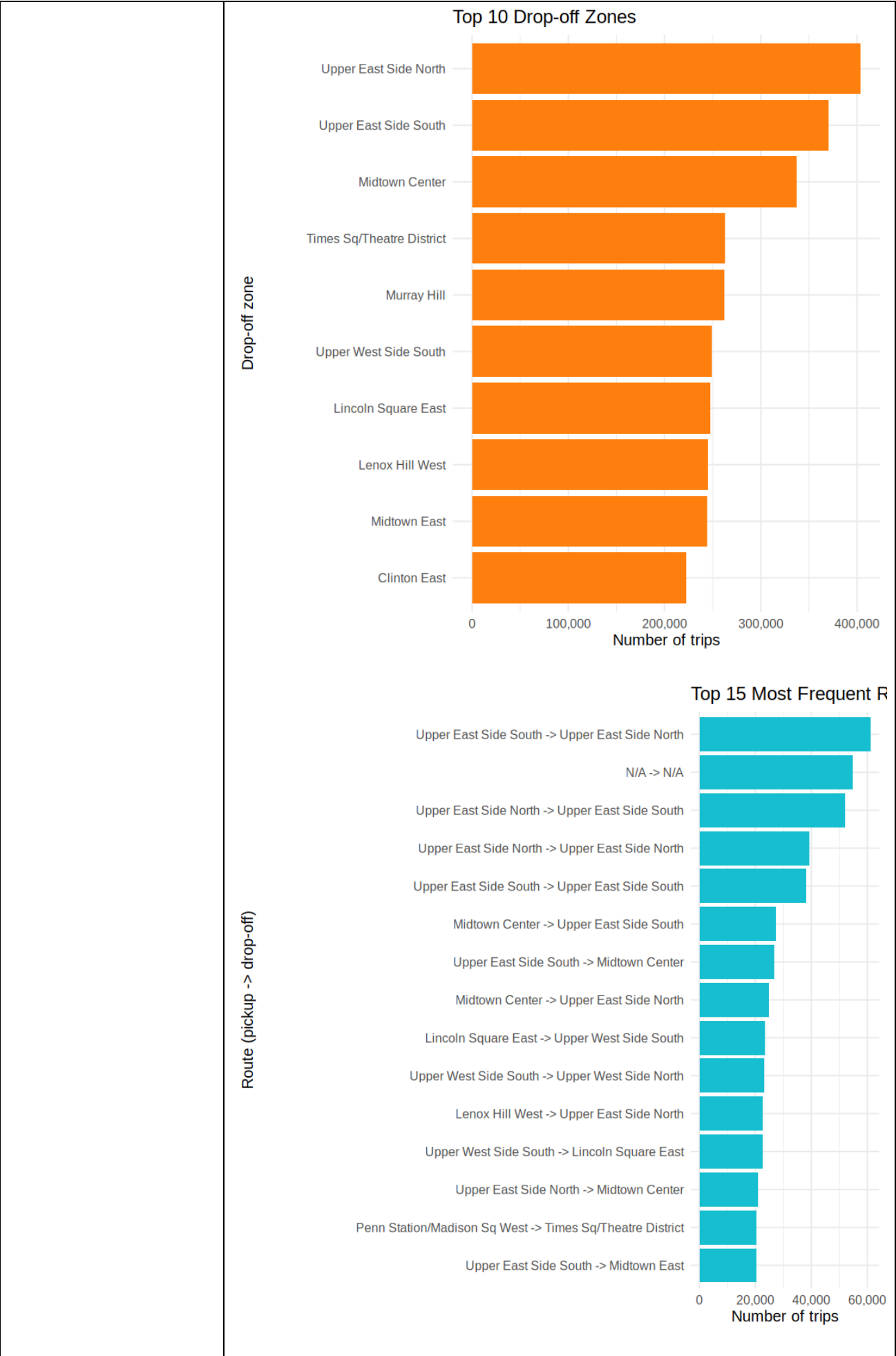


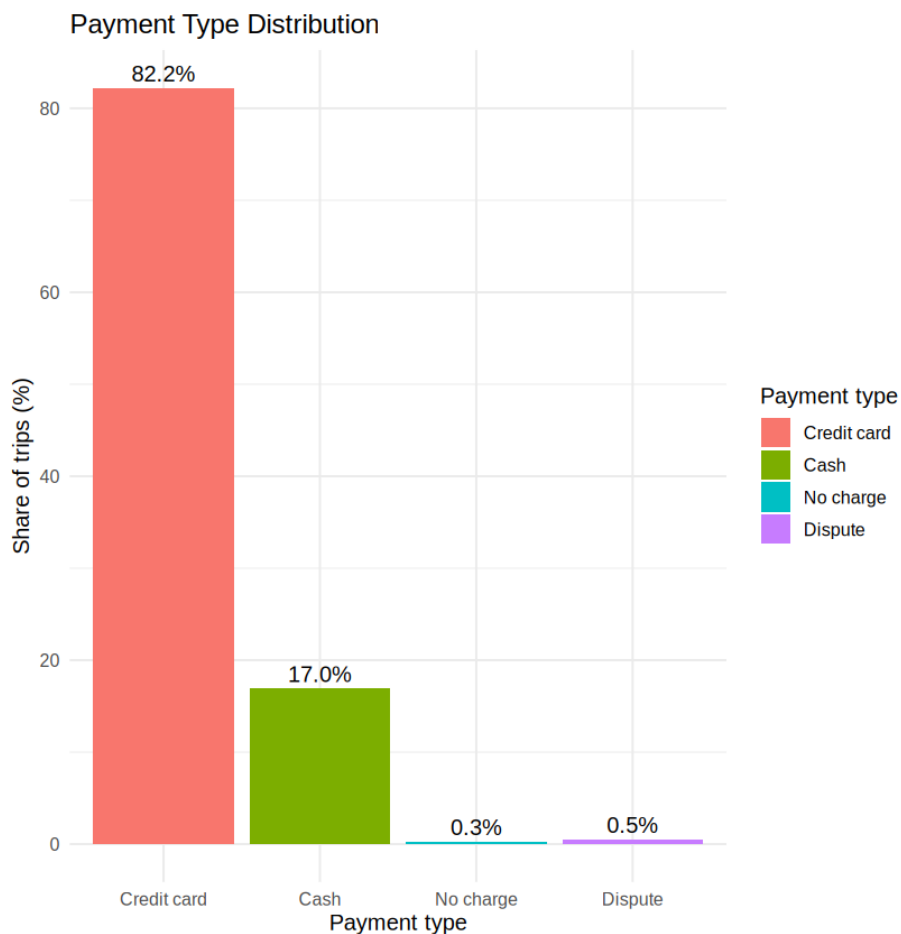
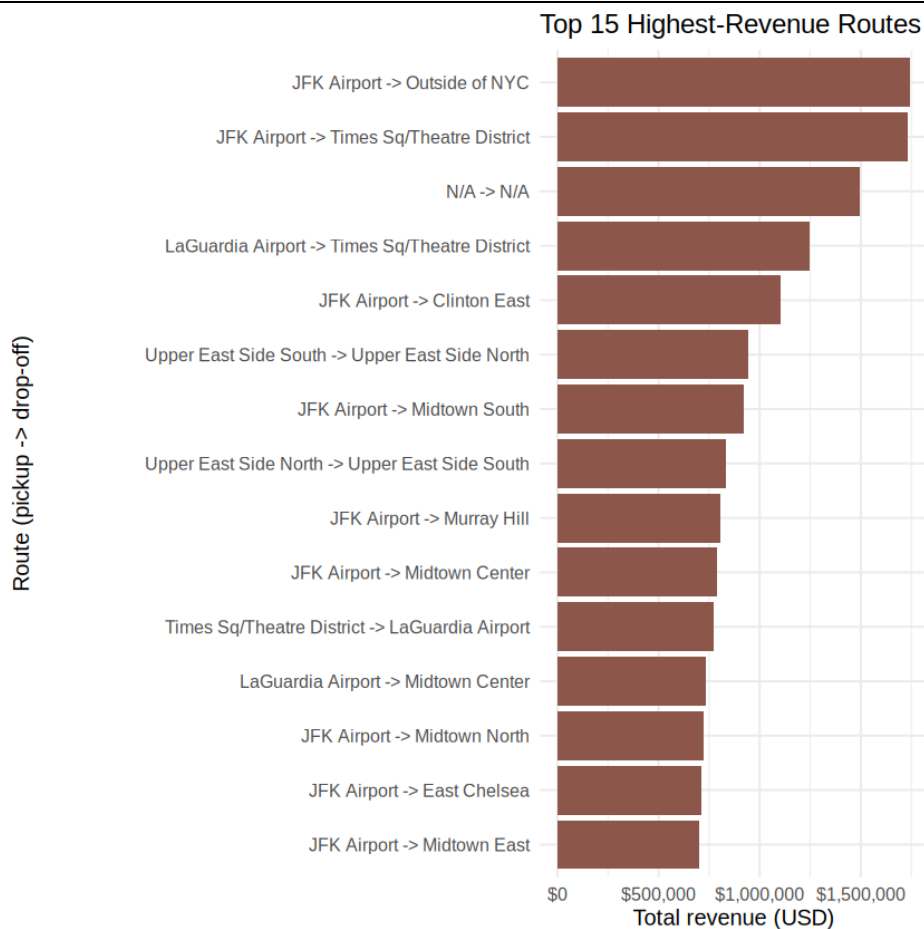
Daily Taxi Revenue



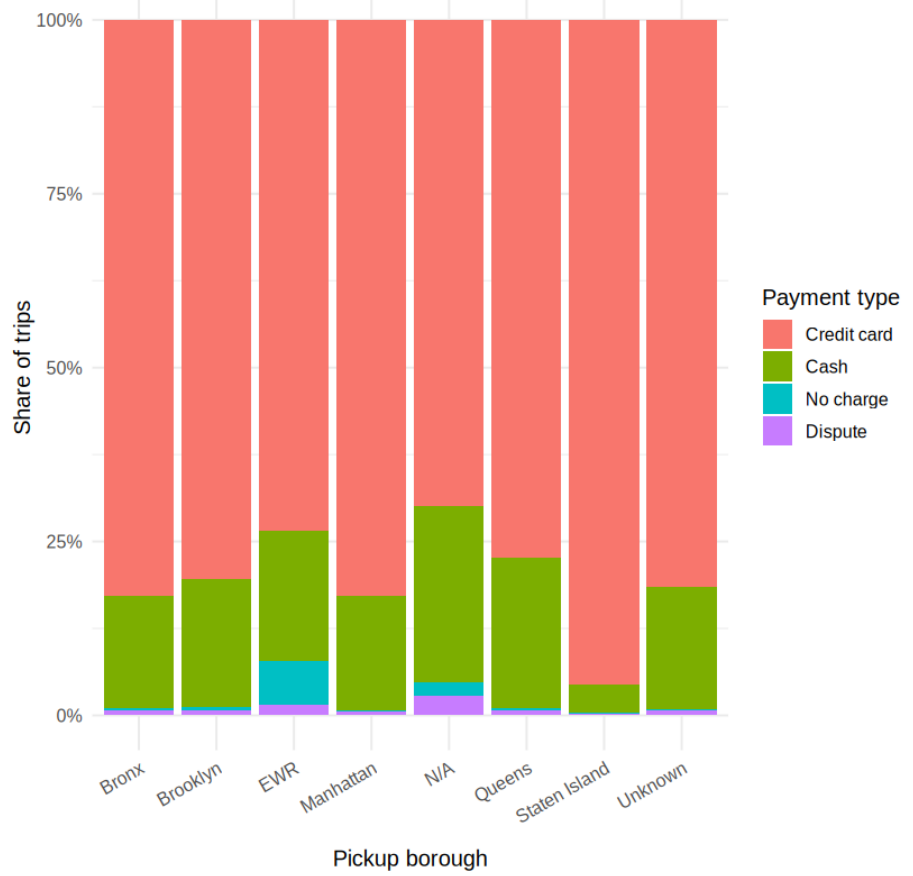
Top 10 Pickup Zones





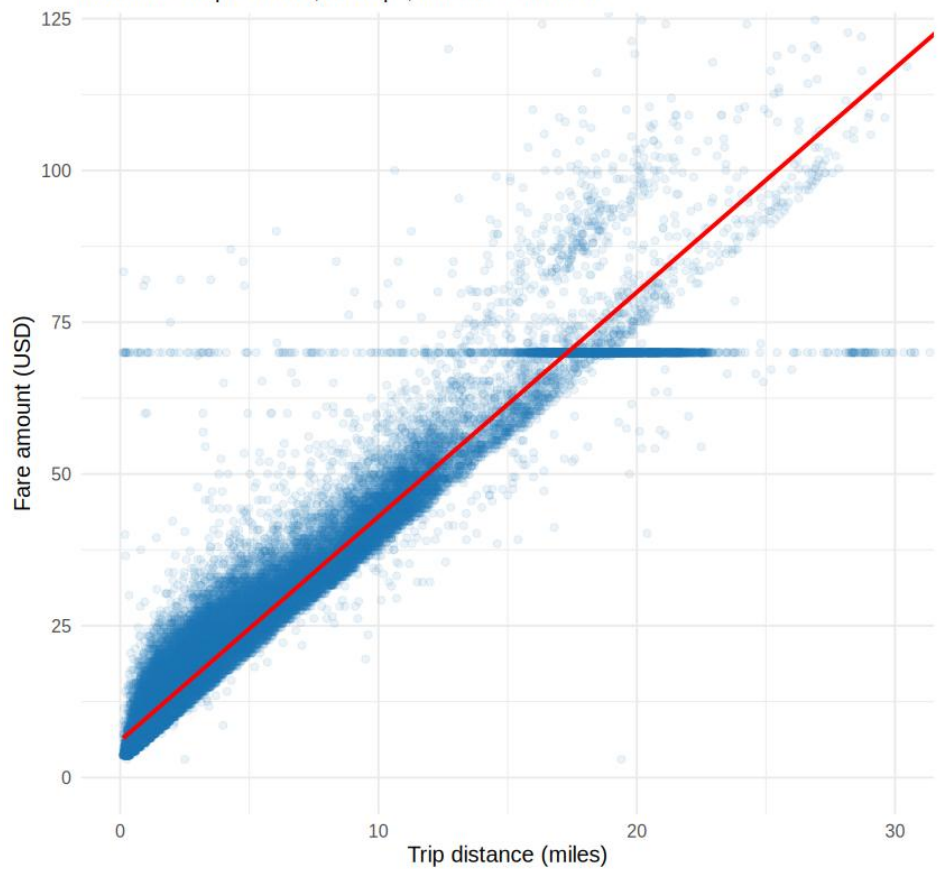


Payment Type by Pickup Borough



Trip Distance vs Fare Amount

Random sample of 100,000 trips; red line = linear fit



	<pre> A data.table: 4 × 4 task method median_ms rel_to_fastest <chr> <chr> <dbl> <dbl> 1 A: Avg fare per hour (9M rows) data_table 324.84 1 2 B: Row-wise tip % (100k rows) data_table_fifelse 1.79 1 3 C: Grouped mean (1M rows) data_table 48.25 1 4 Sequential vs parallel (bootstrap, 21 partitions) parallel_foreach_2_cores 28145.00 1 Speedup of data.table over lapply (task A): 3.8 x Parallel speedup: 1.06 x on 2 cores </pre>
Conclusion	data.table gave the best performance in every benchmark: up to about 650 times faster than apply and 14 times faster than base aggregate, with about one-seventh of the memory of the lapply/map versions on the full dataset. Vectorized operations came a close second, while functional approaches are better kept for heavy per-group work. Parallel processing gave only a 1.06× speedup on 2 cores, which shows that parallelism does not always help when communication overhead is high. For large-scale analytics in R, data.table with vectorized column operations is recommended, with foreach/doParallel added only after measuring that the speedup is worth it.